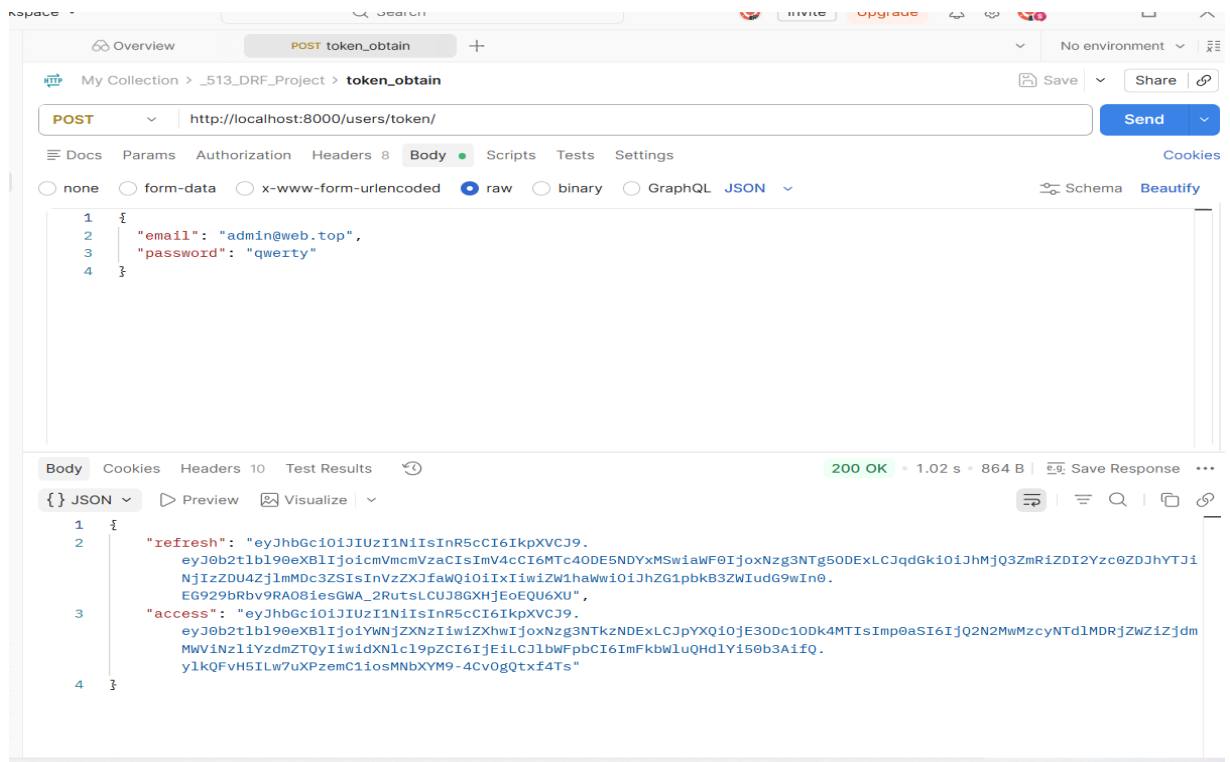
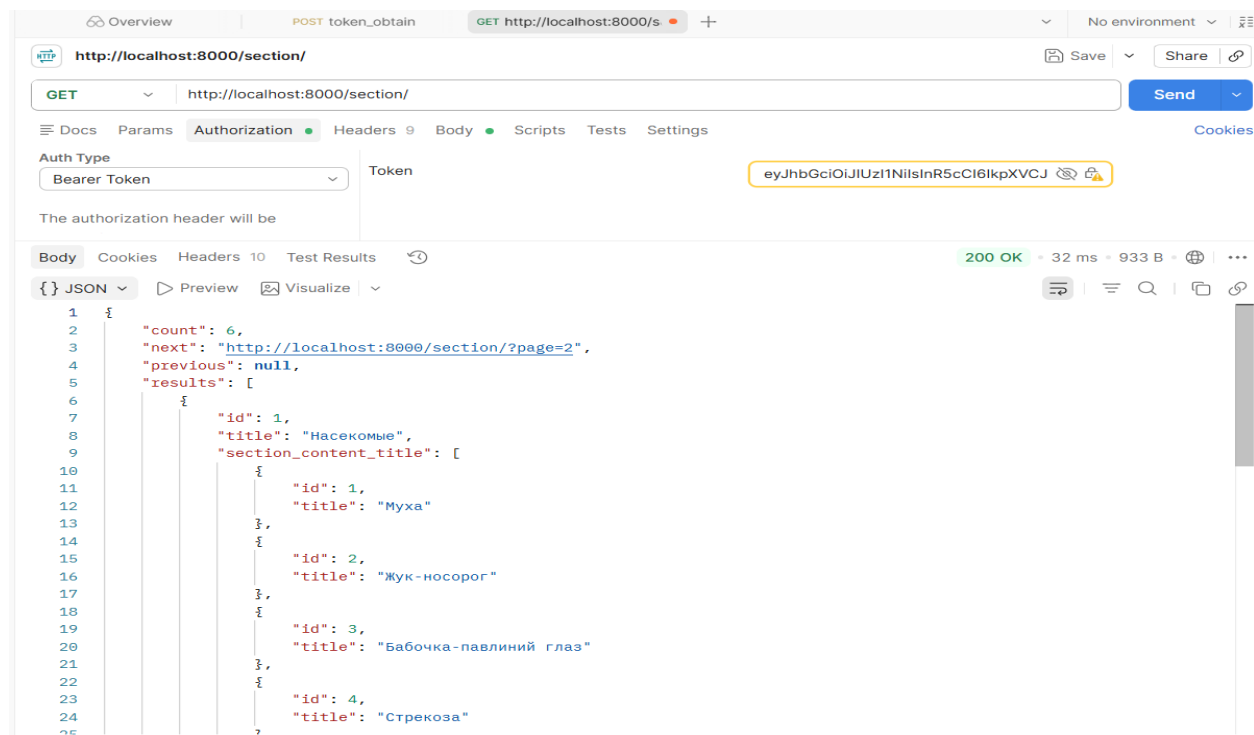


Отчет

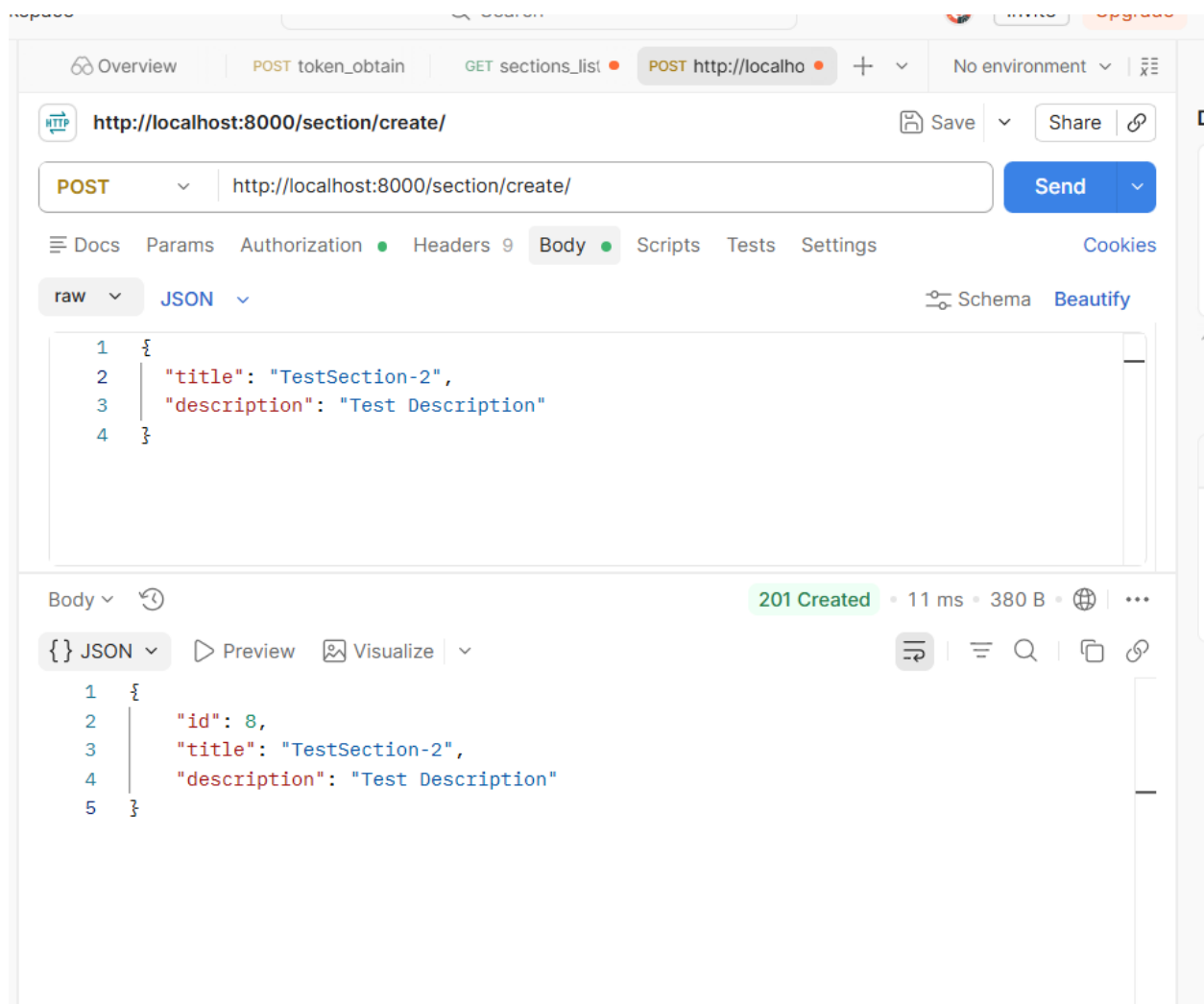
1.Получение токена



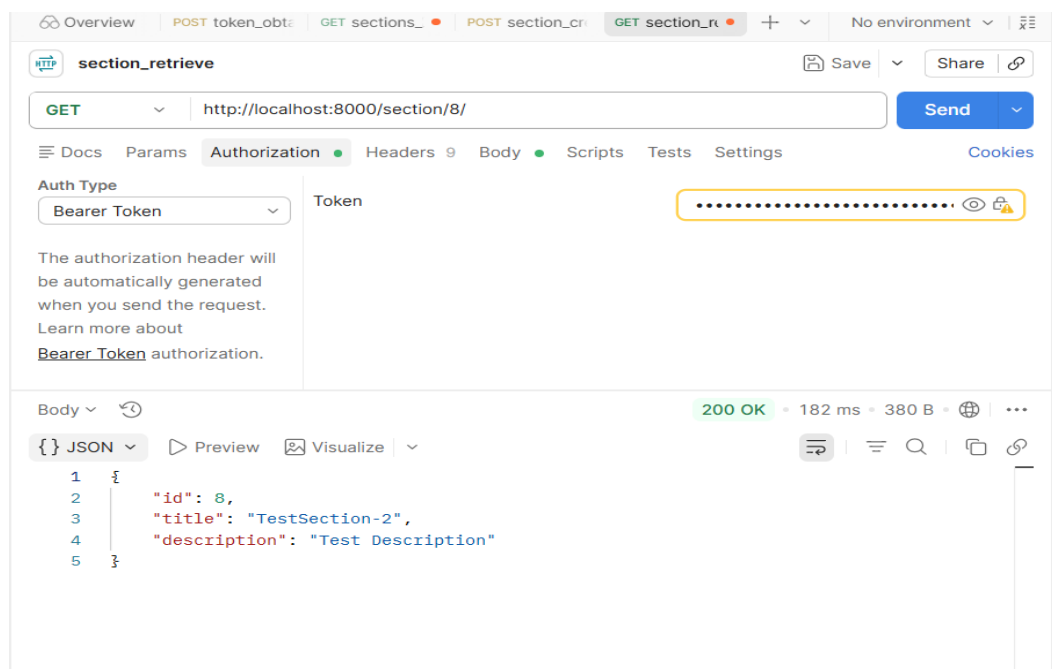
2.1 Запрос на просмотр списка всех разделов



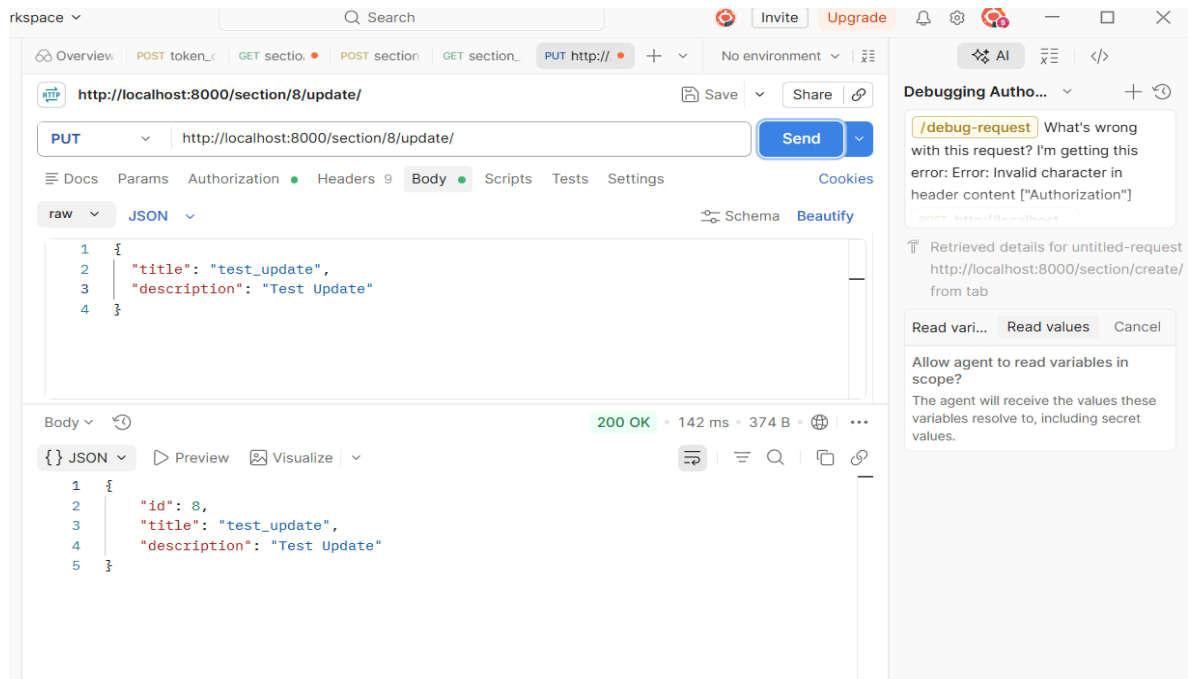
2.2 Создаю раздел в section



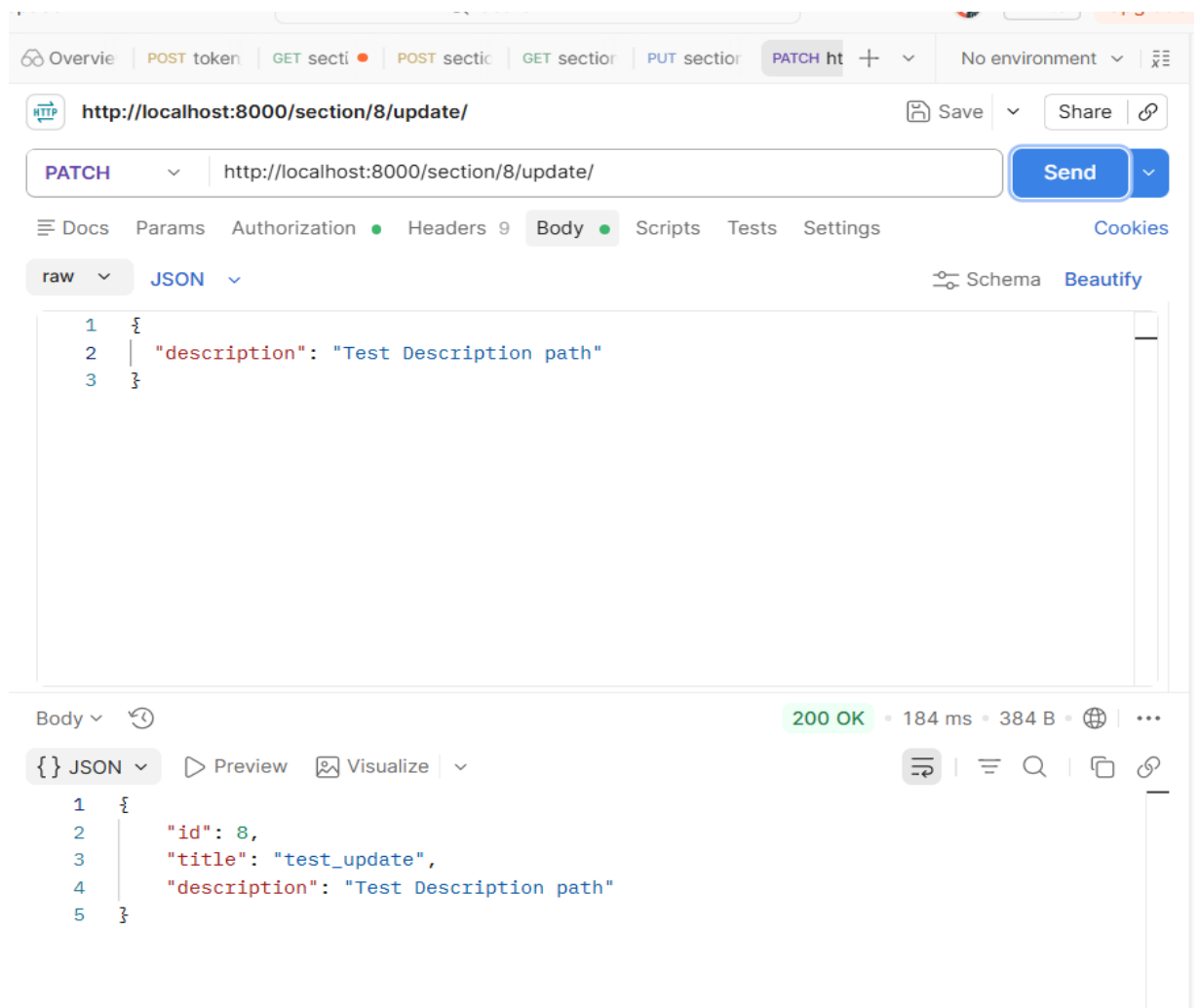
2.3 Просмотр раздела с id=8



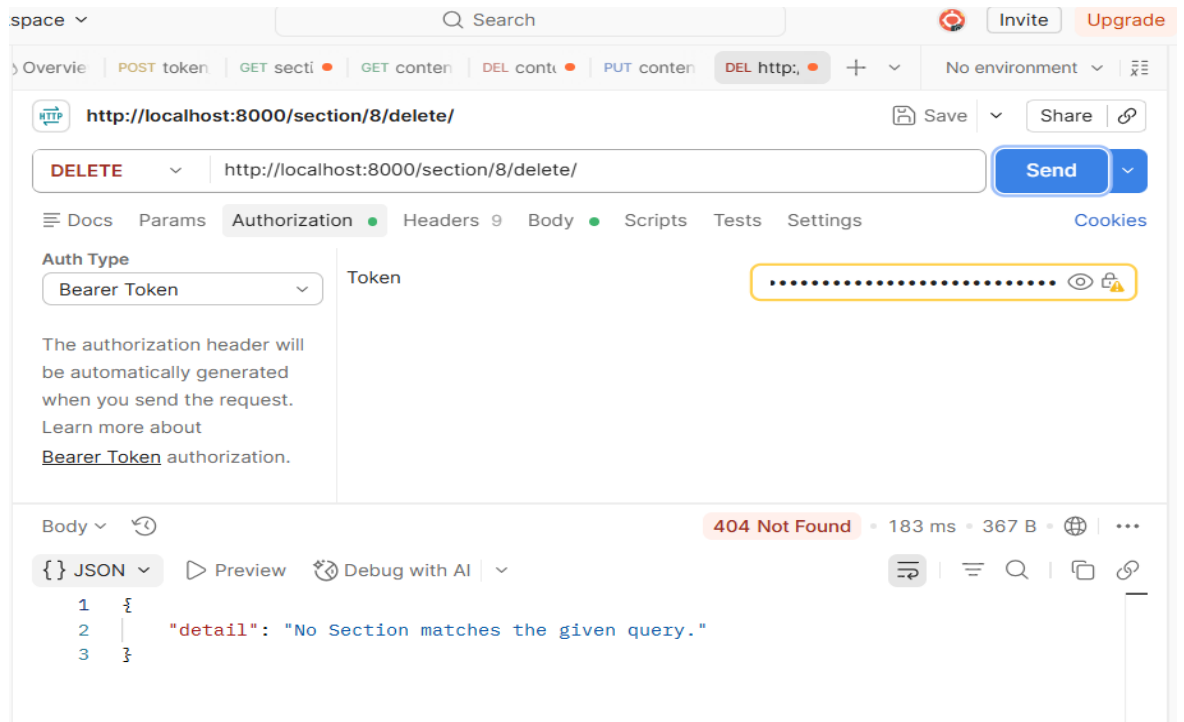
2.4.1 Полное обновление данных в section с id 8



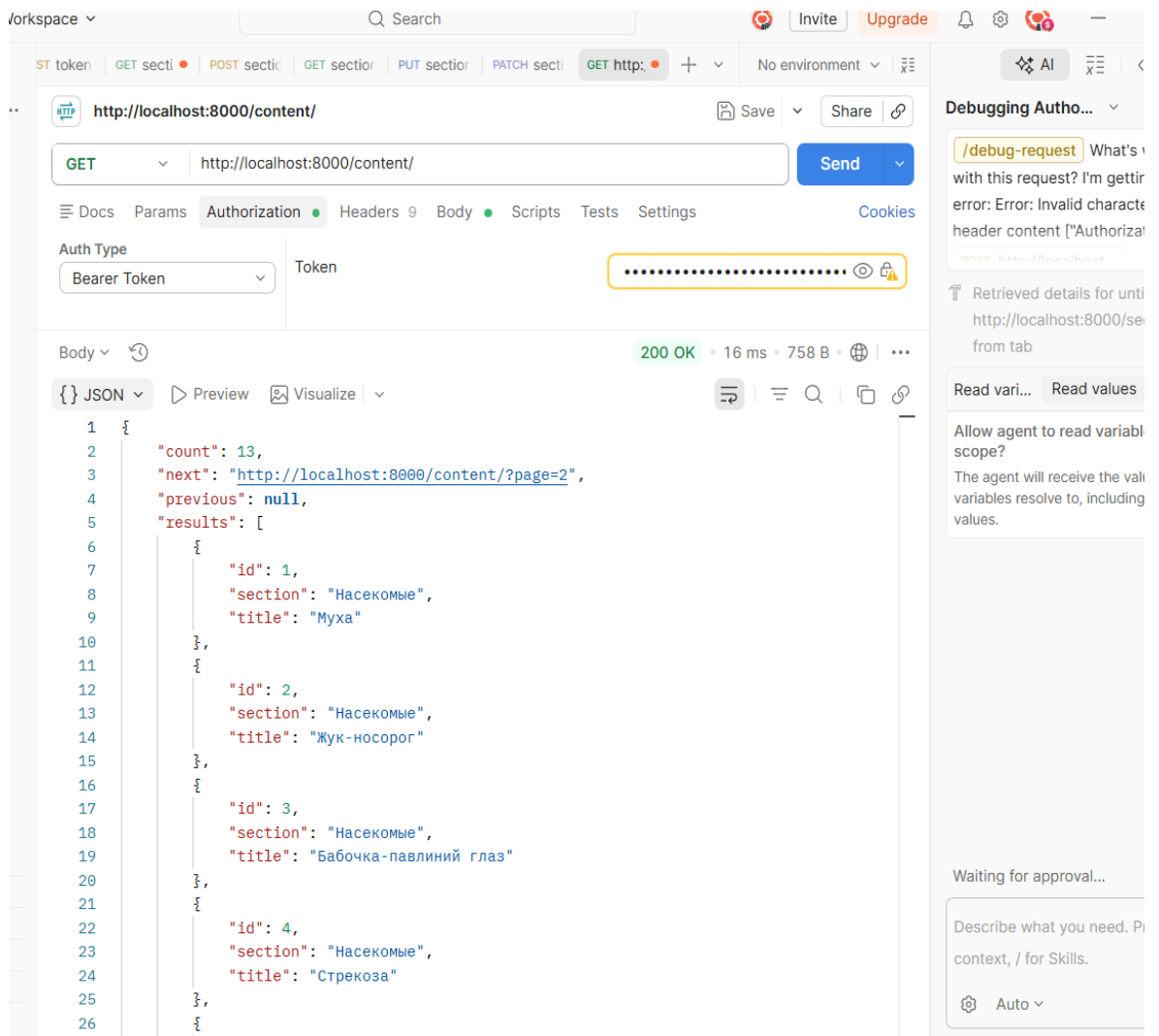
2.4.2 Частичное обновление данных



2.5 Удаление данных в content с id 8



3.1 Просмотр содержимого content



3.2 Создаю новый контент в разделе с ID = 8

The screenshot shows the Postman interface for a POST request to `http://localhost:8000/content/create/`. The request body is a JSON object: `{ "section": 8, "title": "Test Content", "content": "Test Content Create" }`. The response is a 201 Created status with a response time of 188 ms and a body size of 391 B. The response body is a JSON object: `{ "id": 15, "title": "Test Content", "content": "Test Content Create", "section": 8 }`.

```
POST http://localhost:8000/content/create/

{
  "section": 8,
  "title": "Test Content",
  "content": "Test Content Create"
}
```

201 Created • 188 ms • 391 B | Save Response

```
{
  "id": 15,
  "title": "Test Content",
  "content": "Test Content Create",
  "section": 8
}
```

3.3 Получаю данные контента с ID = 15.

The screenshot shows the Postman interface for a PUT request to `http://localhost:8000/content/15/update/`. The request body is a JSON object: `{ "section": 8, "title": "Test Content", "content": "Test Content Create" }`. The response is a 200 OK status with a response time of 186 ms and a body size of 392 B. The response body is a JSON object: `{ "id": 15, "title": "Test Content", "content": "Test Content Create", "section": 8 }`.

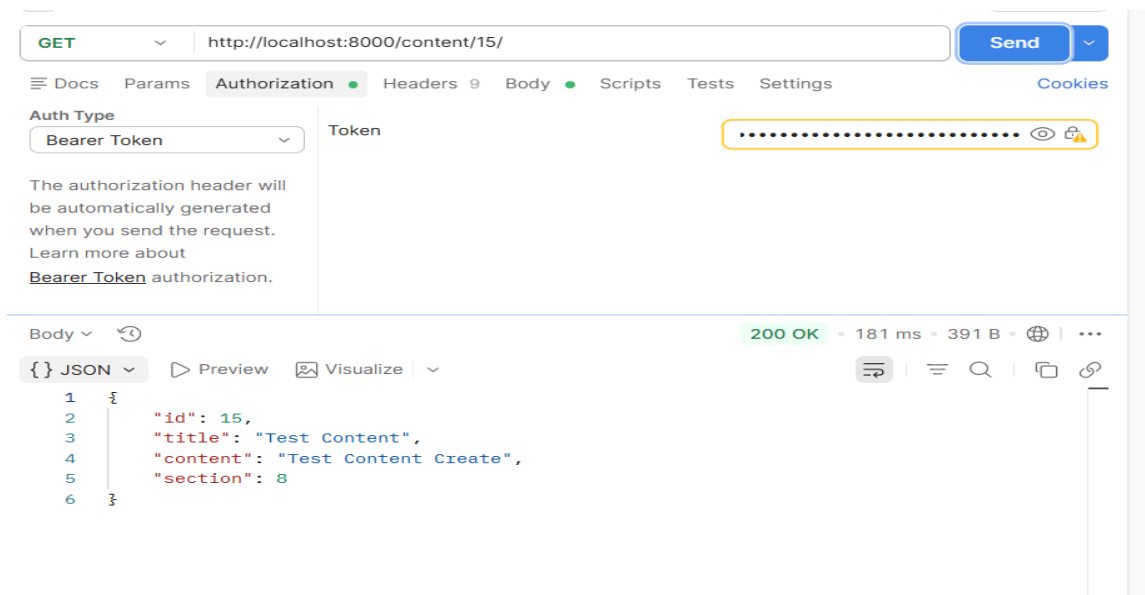
```
PUT http://localhost:8000/content/15/update/

{
  "section": 8,
  "title": "Test Content",
  "content": "Test Content Create"
}
```

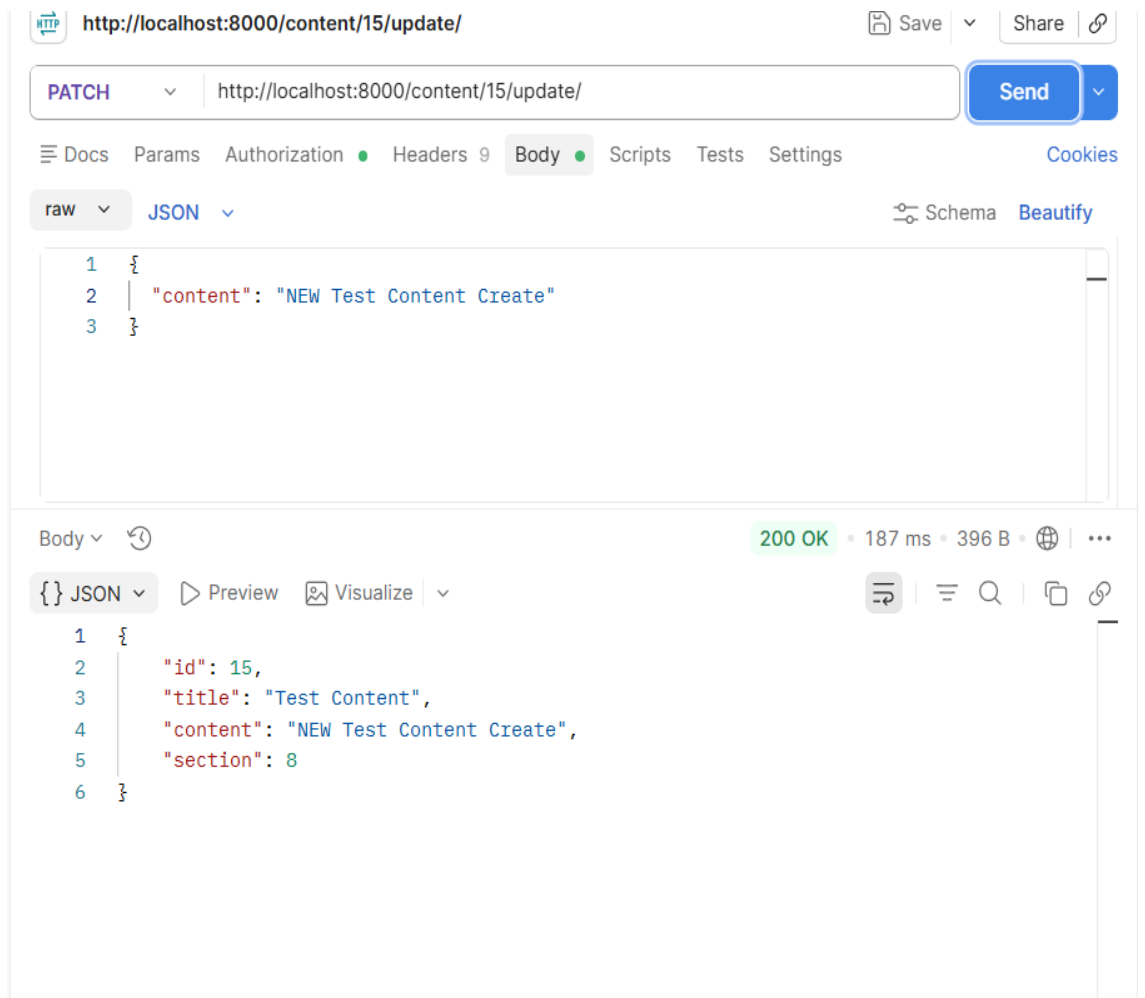
200 OK • 186 ms • 392 B | Save Response

```
{
  "id": 15,
  "title": "Test Content",
  "content": "Test Content Create",
  "section": 8
}
```

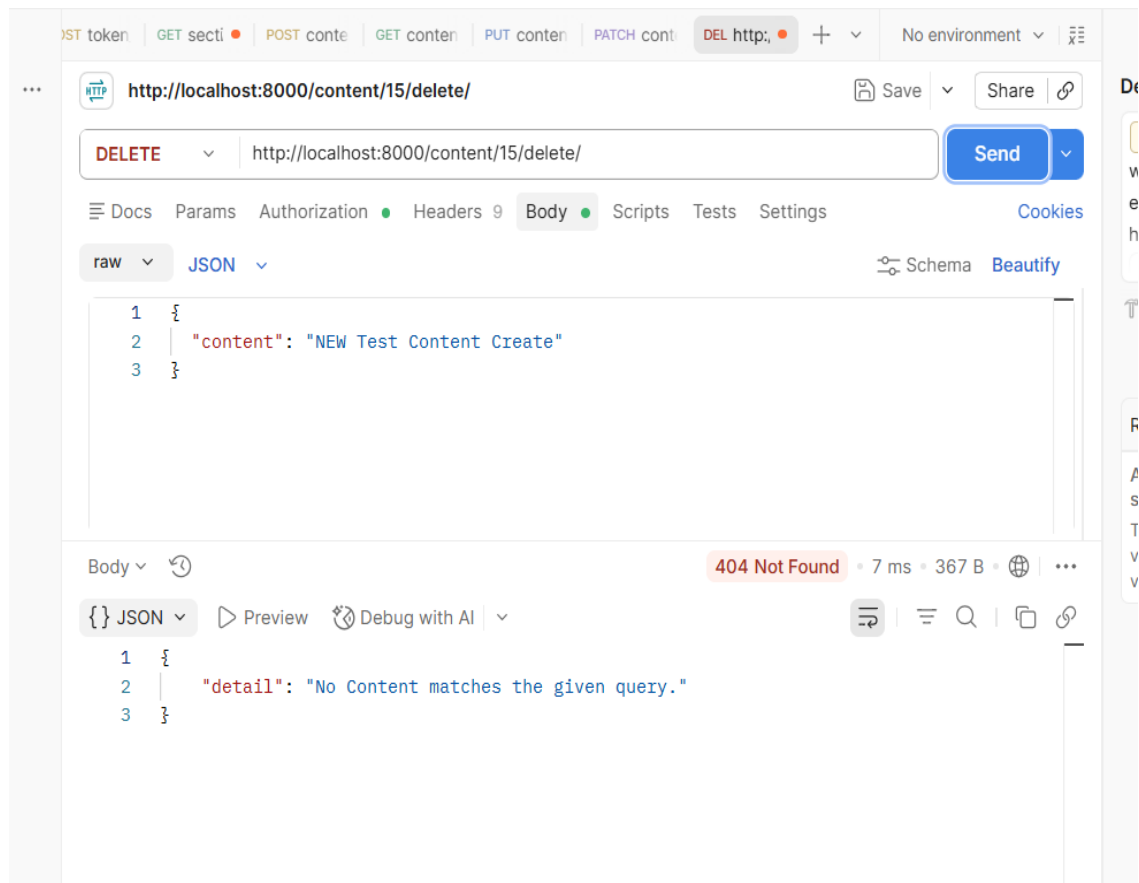
3.4.1 Полное обновление данных в content с id 15



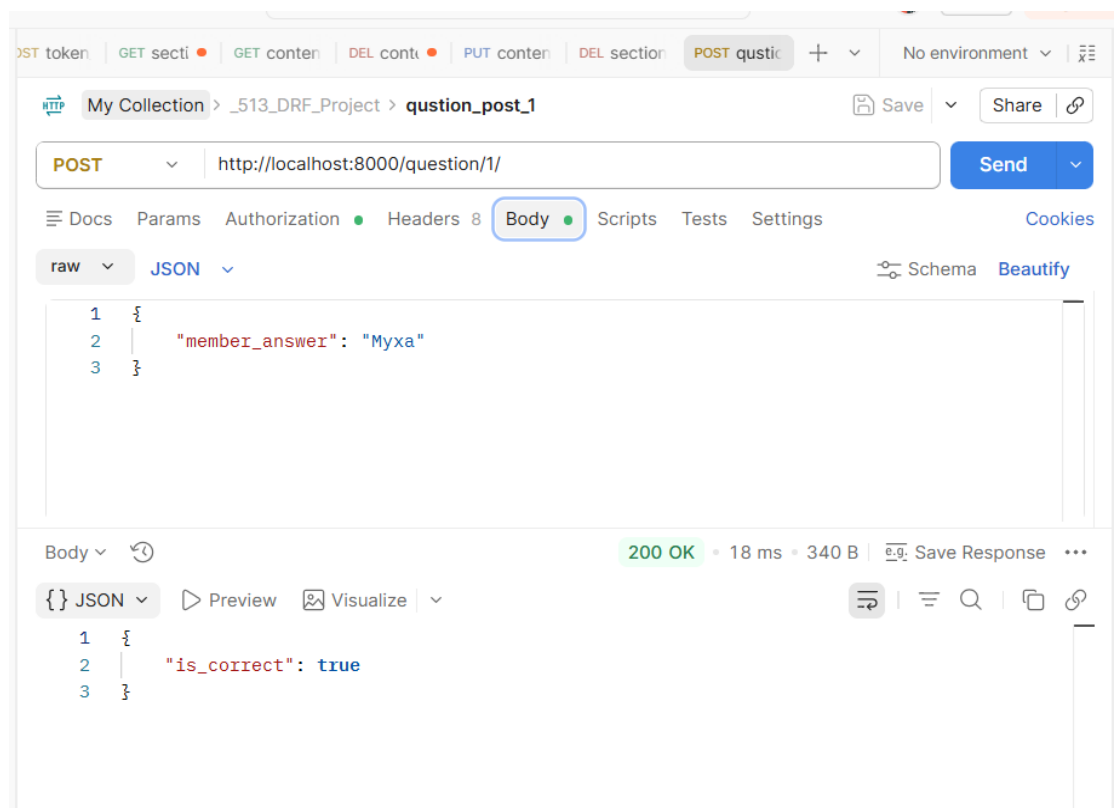
3.4.2 Частичное обновление данных в content с id 15



3.5 Удаление данных в content с id 15



4.1 Запрос для проверки ответа на вопрос.



4.2 Получение конкретного вопроса

HTTP **http://localhost:8000/question/1/** Save Share

GET **http://localhost:8000/question/1/** Send

Docs Params **Authorization** Headers 9 Body Scripts Tests Settings Cookie

Auth Type: Bearer Token Token: [redacted]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body 200 OK • 13 ms • 562 B

```
{
  "id": 1,
  "section": "Насекомые",
  "question": "Какое насекомое из отряда Двукрылые (Diptera) является одним из самых распространённых в мире?",
  "member_answer": null
}
```

4.3 Получил список всех вопросов

Overview POST token PUT conten DEL section POST qustic GET qustic **GET qustic** + No environment Save Share

My Collection > _513_DRF_Project > question_get_all

GET **http://localhost:8000/question/** Send

Docs Params **Params** Authorization Headers 8 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 12 ms • 1.43 KB Save Response

```
{
  "count": 13,
  "next": "http://localhost:8000/question/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "section": "Насекомые",
      "question": "Какое насекомое из отряда Двукрылые (Diptera) является одним из самых распространённых в мире?"
    },
    {
      "id": 2,
      "section": "Насекомые",
      "question": "Какое насекомое из отряда древних летающих насекомых имеет роговидные выросты на голове?"
    },
    {
      "id": 3,
      "section": "Насекомые",
      "question": "Какая бабочка имеет на крыльях характерные глазчатые пятна, напоминающие глаз павлина?"
    },
    {
      "id": 4,
      "section": "Насекомые"
    }
  ]
}
```

5. Все мои запросы

▼ my Collection

▼ _513_DRF_Project

POST token_obtain

GET sections_list

POST section_create

GET section_retrieve

PUT sectiona_update_PUT

PATCH section_update_patch

GET content_list

POST content_create

GET content_retrieve

PUT content_update_put

PATCH content_retrieve_pathc

DEL content_delete

DEL section_delete

POST qustion_post_1

GET question_get_1

GET question_get_all

GET Get data

POST Post data